

不交出原始数据，也能一起把模型练好：用自编码器隐空间嵌入做隐私保护的协同学习

NeurIPS 2022：一种让多方在不暴露原始特征的前提下联合训练机器学习模型的框架

设想这样一个常见场景：两家公司掌握着同一批用户的不同信息。一家有用户的消费流水，另一家有用户的信用记录。如果它们能把各自的特征拼在一起，双方预测同一个目标变量（比如违约概率）都会准得多，决策也会更好。问题在于，这些数据是敏感的。隐私政策和知识产权（Intellectual Property, IP）方面的法律，往往禁止把原始特征直接交给对方，哪怕双方之间的通信通道是安全的。于是合作只能作罢，本该拿到的模型性能提升也随之消失。

这篇来自 Los Andes 大学、Rappi 与 Amazon 的 NeurIPS 2022 论文提出了一个直接的思路：既然不能共享原始数据，那就共享数据的一种「模糊化」表示。具体做法是，每一方先用自编码器（Autoencoder）把自己的表格数据编码成一段隐空间（latent space）表示，然后只把这段编码后的嵌入拿出来共享。多方再按照共同的观测 ID 把各自的嵌入拼接起来，用拼好的嵌入去训练一个下游的有监督模型。原始特征始终留在各自手里，从没离开过。

作者在三个公开基准上验证了这个框架。核心结论是：用共享的隐空间嵌入替换原始特征后，下游模型的性能损失小于 10 个百分点（percentage points, pp），而自编码器的重建误差介于 5% 到 11% 之间。换句话说，敏感特征被藏了起来，预测能力却几乎原封不动地保留了下来。

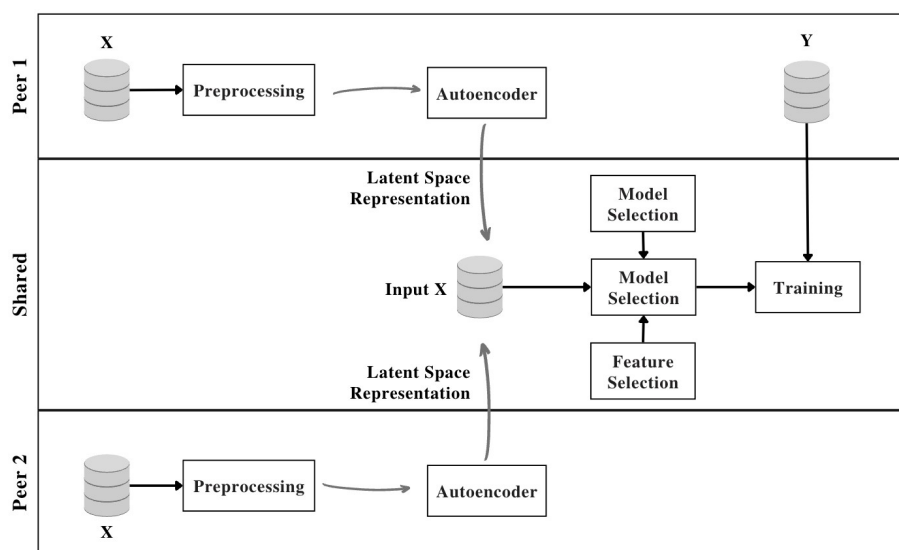


图 1 隐私保护机器学习框架。两方各自把数据库编码成隐空间表示，只把这份表示送到一个共享的数据源；随后在共享层完成模型选择、特征选择与训练。原始特征 X 从不离开各自一方。

论文地址：<https://arxiv.org/abs/2211.05717>

为什么已有的隐私方案还不够用

学术界和工业界其实已经积累了不少隐私保护策略，但每一种都有它的短板。加密类方法（例如同态加密，homomorphic encryption）安全性很强，可对技术栈的要求高，在真实业务里部署起来相当吃力。联邦学习（federated learning）把训练分散到各个设备上，但它默认每一方持有的是同一类特征，因而处理不了「两方各自贡献不同特征」这种横向拼接的情形。差分隐私（differential privacy）靠往数据里加噪声来掩盖单个取值，代价是可能明显降低数据可用性。而主成分分析（PCA）这类线性降维虽然也能起到模糊化作用，却容易丢掉特征之间重要的非线性关系。

这些方法大多只保护「传输中」的数据，或是改变了单条观测的分布模式，又或者维护成本很高。它们共同留下的空白，正是这篇论文想填补的：能不能用深度表示学习，把数据编码成一种既可以放心共享、又保留了预测结构的形式？

方法：在数据共享流程里插入一步编码

这个框架的巧妙之处在于，它并没有推翻常规的数据共享流程，而只是在中间插入了一步。原本大家要做的是把各自的原始数据集合并；现在改成，每一方先让自己的特征过一遍自编码器，得到一段隐空间表示，也就是一份可以拿出去共享的「模糊化」数据。多方随后用共同的观测 ID 把这些表示对齐拼接，再在拼好的嵌入上训练同一个有监督模型，去预测同一个目标。

为了让结论反映的是框架本身、而不是某次精细的网络调参，作者在所有实验里都固定使用同一个自编码器结构。编码器有四层，把 N 维输入特征依次压到隐藏维度 128、64、40，最后落到一个大小为 M 的嵌入；解码器则镜像地把 M 还原回 N 。每一层都用 ReLU 激活，损失函数取平均绝对误差（Mean Absolute Error, MAE），优化器为 Adam，学习率固定在 0.0001。

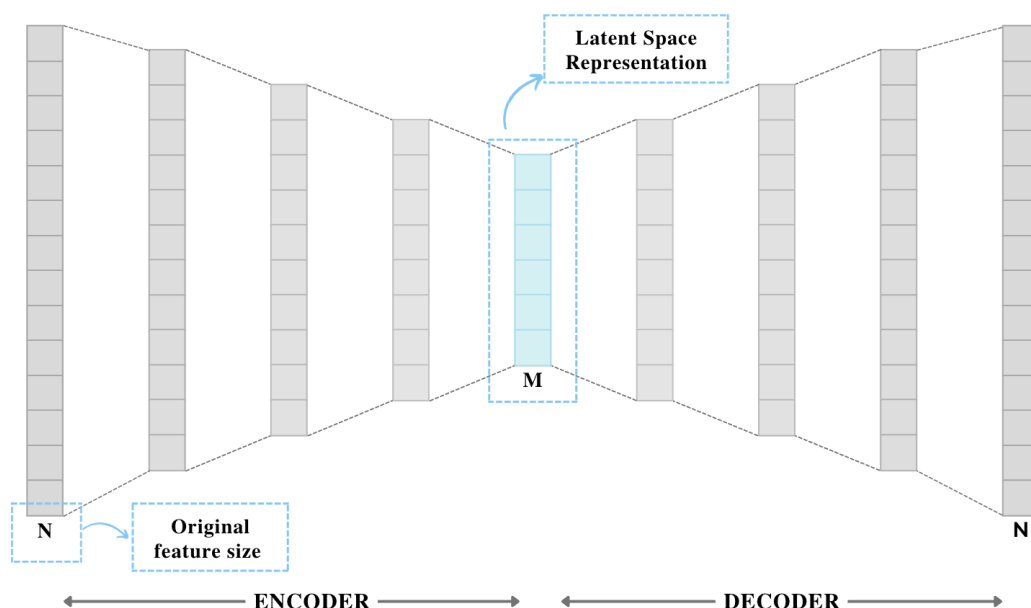


图 3 自编码器结构。编码器把原始特征维度 N 逐层压缩到隐空间维度 M ，解码器再把 M 还原回 N ；中间那段大小为 M 的向量，就是拿去共享的隐空间表示。

五个场景：把每个设计选择拆开来

论文把实验组织成一次跨五个场景的消融，让每一个设计决策都能被单独看清楚。场景 0 是不加任何隐私保护的原始数据基线，它给出的性能，就是后续所有场景想尽量守住的上限。场景 1 用一个共享的自编码器去编码一份合并后的数据集。场景 2 则模拟真正的两方协作：每一方各有一个自己的自编码器，两份嵌入再拼到一起。

场景 3 和场景 4 分别是场景 1、场景 2 的「非朴素」（non-naive）版本。所谓非朴素，是指把自编码器改造成一个多任务网络：它在编码的同时，还要预测那个下游的目标变量。这样一来，隐空间就不再是盲目地追求重建，而是被下游任务「牵引」着成形。图 2 用三种颜色标出了不同场景下数据的流向。

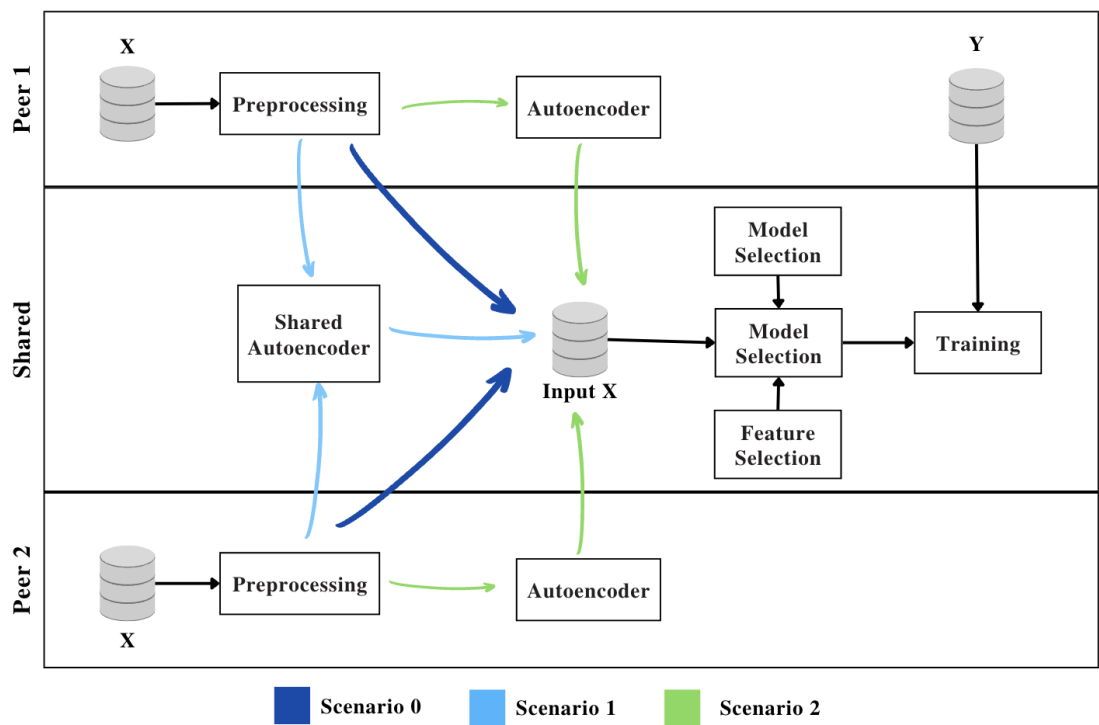


图 2 训练阶段的通用自编码器结构与本文探索的各个场景。深蓝为场景 0（原始基线），浅蓝为场景 1（单个共享自编码器），绿色为场景 2（每一方各自的自编码器）。非朴素的场景 3、4 在此基础上让自编码器额外预测目标变量。

三个基准数据集

为了在尽量贴近真实、又足够多样的条件下检验框架，作者选了三个公开数据集。它们在下

游任务、维度和特征类型上都刻意拉开了差距，以此考察编码方法的稳健性与可扩展性。所有实验都跑在一套并不豪华的设备上：一颗双核 2.30 GHz 的 Intel Xeon CPU、一块 12 GB 显存的 Nvidia K80 或 T4 GPU，以及 12 GB 内存。

表 1 三个公开基准数据集及其下游任务。

数据集	观测数	特征数	机器学习任务
-----	-----	-----	--------

House Pricing（房价）	21,613	12	回归
MNIST Numbers（手写数字）	35,000	784	多分类
Buzz in Social Media（社交热度）	87,488	77	回归

结果：藏起原始特征，几乎不掉性能

最关键的发现是：把共享内容从原始特征换成编码后的表示，几乎不损害预测性能。在 House Pricing 上，原始数据基线的测试集 R^2 约为 90.29%，而模拟两方、各用一个自编码器的场景 2 仍能达到 89.33%，在数据已被模糊化的前提下，这个跌幅几乎可以忽略。在 Buzz in Social Media 上，基线 R^2 约 96.19%，编码后的各场景则稳定停留在八十几到九十出头的水平。对 MNIST 数字分类，测试准确率从基线的 92% 下降到编码后的 84% 到 88%。

更值得注意的是那两个非朴素场景。让自编码器被目标变量牵引之后，它们把前面丢掉的性能补回了一大截：在 Buzz 上，测试 R^2 从场景 1 的 91.55% 回升到场景 3 的 94.03%；在 MNIST 上，准确率从 88% 回到 91%。这说明「用下游任务来塑造隐空间」确实有效。下表汇总了三个数据集在各场景下的测试集表现。

表 2 三个数据集在五个场景下的测试集表现（House Pricing 与 Buzz 报告 R^2 ，MNIST 报告准确率）。

场景	House Pricing (R^2)	MNIST (准确率)	Buzz (R^2)
场景 0：原始基线	90.29%	92%	96.19%
场景 1：单个共享自编码器	84.27%	88%	91.55%
场景 2：各自的自编码器	89.33%	84%	89.01%
场景 3：非朴素共享	89.21%	91%	94.03%
场景 4：非朴素各自	87.36%	84%	87.90%

几个值得记住的数字

把整篇工作的影响浓缩成几个数字：在所有用例上，用共享隐空间表示替换原始特征后，下游性能的下降都小于 10 个百分点，而自编码器的重建误差落在 5% 到 11% 之间。在 House Pricing 上，测试 R^2 只从基线的 90.29% 降到两方场景的 89.33%。在 Buzz 上，非朴素的共享自编码器（场景 3）达到 94.03% 的 R^2 ，对应基线为 96.19%。此外，共享自编码器对每个特征平均能正确估计 96% 到 98% 的观测（以 MAPE 不超过 5% 计），说明嵌入确实保住了数据的核心结构。

小结与展望

结论很朴素：与其交换原始的敏感数据，协作的组织完全可以只共享自编码器产出的隐空间嵌入。这既让原始特征保持私密，又在三个基准上把大部分预测能力保留了下来，性能损失小于

10 个百分点。而只要让自编码器「知道」下游任务是什么（即非朴素设计），这个差距还能进一步缩小。

作者也指明了下一步的方向：为每个数据集定制专门的自编码器，以及引入对「隐私强度」本身的正式度量。目前这套框架更多是从「保留预测能力」的角度给出证据，至于编码究竟在多大程度上抵御了逆向还原，仍值得用更严格的隐私度量去回答。值得强调的是，整套流程只在常规的数据共享管线里多加了一步编码，既不需要改造下游模型，也不依赖专门的加密基础设施，因此迁移成本很低。对于金融风控、医疗、营销这些既渴望联合建模、又受制于合规红线的场景，这种「只交换嵌入、不交换原始特征」的做法，提供了一条容易上手、也便于逐步验证的路径。对于真正想在组织间落地协同建模的团队来说，这是一个务实、门槛不高的起点。